Digital Logic Design Report

Overdrive

Group Members:
Azyan Ahmed, Arham Hussain,
Bareera Junaid, Syeda Fatima Alam

Instructor: Farhan Khan

Spring 25'

Date: 11th May, 2025

TABLE OF CONTENTS

1. Introduction

Our project “Overdrive” focuses on designing a fully functional, two-player arcade-style racing game using Verilog on the Basys 3 FPGA board. The game features a VGA display and offers multiple play modes, including lap-based and time-based races. Designed to recreate the feel of a traditional arcade experience, players control their cars on a simulated track, navigating checkpoints and completing laps, with real-time performance tracked on-screen to enhance competition. The primary aim is to deliver a complete retro arcade experience by integrating physical controls and competitive two-player mechanics.

A standout feature is the creation of a physical arcade cabinet that houses the system and incorporates hardware-based controls. Mimicking a real arcade machine, it includes a joystick for steering and acceleration pedals. To enhance the immersive feel and better mimic real-world driving, we integrated IR sensors within the pedal system, allowing players to control acceleration naturally through physical foot pressure. This provides a tactile and responsive input mechanism that moves beyond simple button presses, delivering an authentic, hands-on driving feel.

Ultimately, the project's goal extends beyond just building a game to designing a complete physical arcade gaming system. By combining digital gameplay with the tactile satisfaction of real-world controls and bridging hardware and software through carefully designed modules and sensor-driven interfaces, we have successfully developed an entertaining, compact, and fully interactive arcade-style gaming experience.

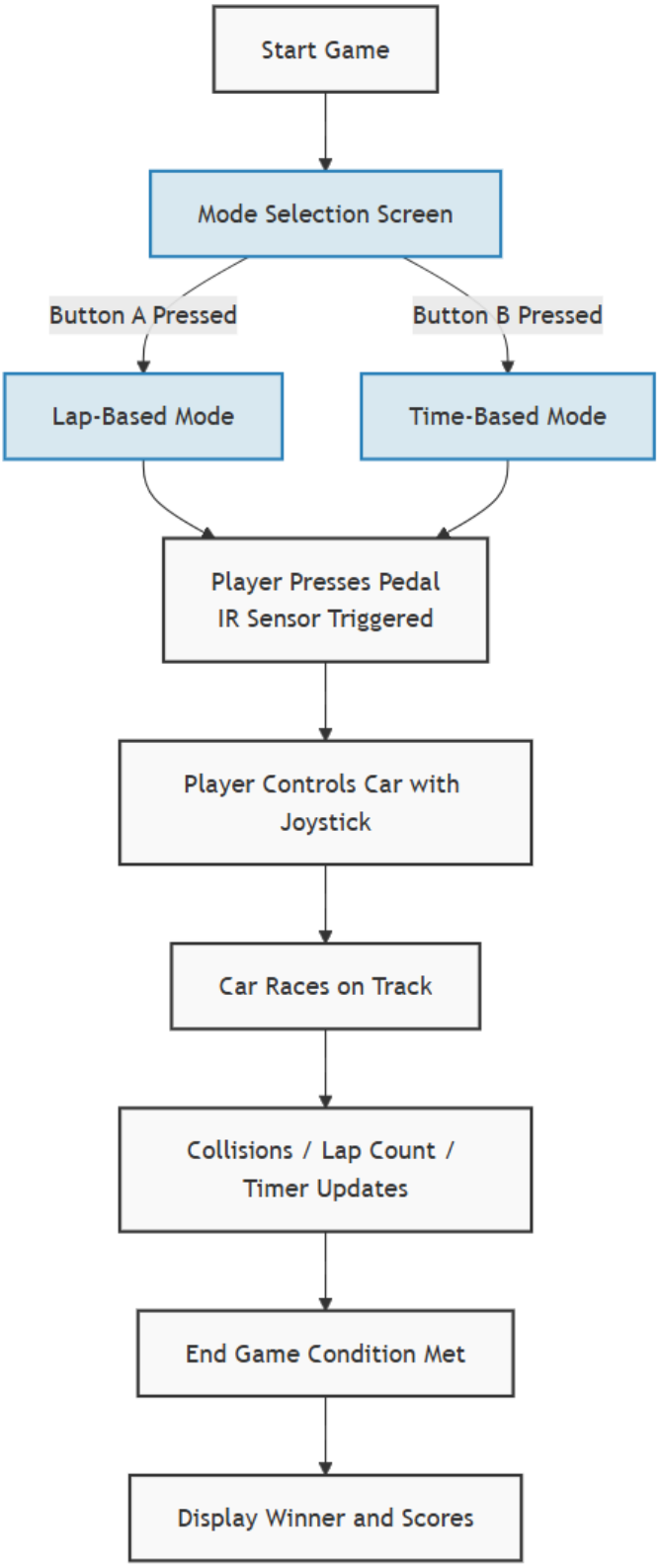
2. System Overview

Our racing arcade machine features:

- A wooden arcade cabinet
- VGA display for real-time graphics
- Pedal input system with IR sensors
- Joystick controller
- Green and red buttons
- Basys 3 FPGA handling logic, game states, and rendering
- Two game modes: Lap-Based and Time-Based

The game allows two players to compete in a racing match by either completing the most laps or having the highest score when time runs out. Movement is controlled using a joystick and the pedal (via IR sensor), and VGA output handles visuals like the track, cars, and UI.

3: USER FLOW DIAGRAM :



x`

4. Hardware Components and Input Processing

Our car racing game implemented on the Basys 3 FPGA relies on a specific set of hardware components to provide the physical interface and processing power necessary for gameplay. This setup includes the game's physical structure, display, and the core input mechanisms.

The entire system is housed within an **Arcade Cabinet**, specifically designed for an average user, creating an immersive, arcade-like experience. Visual feedback is provided through a standard **VGA Computer Monitor**, connected to the FPGA, which displays the game's graphics and interface.

At the heart of the system is the **Basys 3 FPGA Board**. This board serves as the central processing unit, executing the game logic, handling all input signals, and generating the video output for the monitor. The FPGA and connected components are powered by a reliable **Power Supply**, either via USB or an external 5V source.

INPUTS:

- **Joystick:** This serves as a primary controller, predominantly used for directional control within the game and for navigating menus or selecting game modes.
- **Buttons:** There are two buttons – a green one and a red one for different actions.
- **Pedal + IR Sensor Setup:** Foot pedals simulate acceleration. Attached to these pedals are **IR Sensors** that accurately detect when a pedal is pressed. This system provides tactile feedback for speed control.

To ensure the reliability of inputs from these physical devices, several processing blocks are implemented on the FPGA:

- **Debounce Modules:** Physical buttons and switches often produce noisy signals when pressed or released (bouncing). Debounce modules are implemented for inputs like the buttons (green, and red) and signals derived from the pedals to filter out these transient signals, ensuring that each press is registered cleanly as a single event.
- **IR Sensor Module:** This dedicated module processes the signals from the IR sensors attached to the pedals. It incorporates debouncing logic and edge detection to reliably determine the precise moment a pedal is pressed or released, providing accurate acceleration and braking control.
- **Joystick Module:** The module was provided by our instructor and we made changes to it for directional input.

- **Mode Selection Inputs:** Logic is implemented to interpret the signals from the green and red buttons on the joystick, allowing the player to select different game modes or make confirmations within the game interface.

```
module debouncer (
    input wire clk,          // 100MHz clock
    input wire btn,          // Raw button input
    output reg btn_clean     // Debounced output
);
    reg [19:0] count = 0;    // Counter for ~10ms
    reg btn_reg = 0;         // Previous button state

    always @(posedge clk) begin
        btn_reg <= btn;
        if (btn_reg != btn_clean) begin
            count <= count + 1;
            if (count == 20'hFFFFF) begin // ~10ms
                btn_clean <= btn_reg;
                count <= 0;
            end
        end
        else
            count <= 0;
    end
endmodule
```

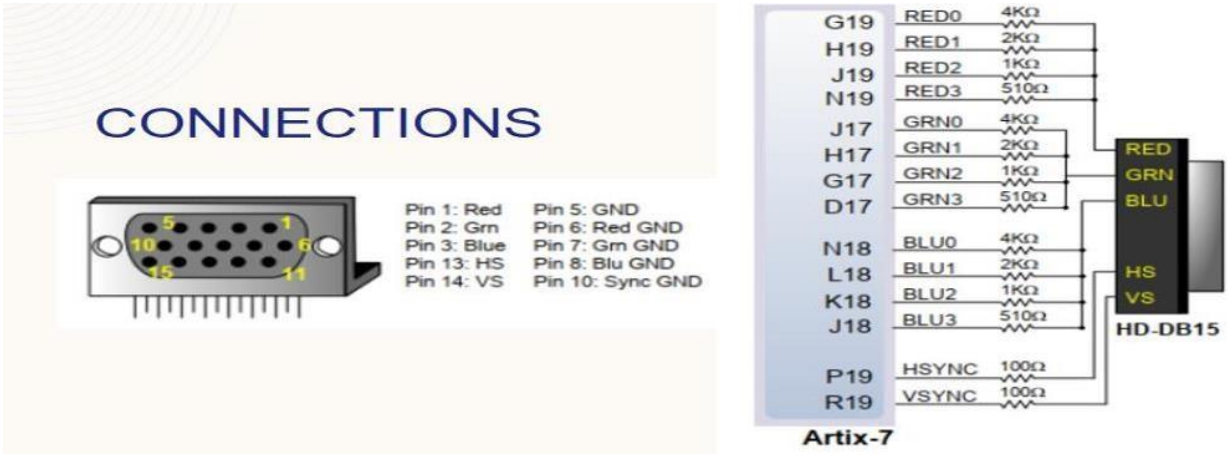
```
module ir_detector(
    input wire clk,             // System clock
    input wire reset_n,         // Active-low reset
    input wire ir_sensor_in,    // IR sensor input from PMOD
    output reg led_out          // LED output
);

    // Parameters (adjust as needed for your specific hardware)
    parameter ACTIVE_HIGH = 1;          // Set to 0 if using active-low I
    parameter DEBOUNCE_CYCLES = 1000;   // Debounce period (clock cycles)

    // Internal registers
    reg [15:0] debounce_counter;
    reg ir_stable;
    reg ir_previous;

    // Normalize the IR sensor input based on active level
    wire ir_sensor = ACTIVE_HIGH ? ir_sensor_in : ~ir_sensor_in;

    // Main detection logic with debouncing
    always @(posedge clk or negedge reset_n) begin
        if (!reset_n) begin
            debounce_counter <= 0;
            ir_stable <= 0;
            ir_previous <= 0;
            led_out <= 0;
        end else begin
            // Store the previous value for edge detection
            ir previous <= ir sensor;
```

```
21  module Joystick(
22      input CLK100MHZ,
23      input vauxp6,
24      input vauxn6,
25      input vauxp7,
26      input vauxn7,
27      input vauxp15,
28      input vauxn15,
29      input vauxp14,
30      input vauxn14,
31      output reg [7:0] Directions
32    );
33
34      wire enable;
35      wire ready;
36      wire [15:0] data;
37      reg [6:0] Address in;
```

5. Output Block Details

- **VGA Output**: 640x480 resolution.
- **Car Sprites**: Dynamic rendering with directional adjustments and sprite rotation.
- **Text Display**: ASCII ROM driven display of scores, timer, and winner.

- The display screen is generated using the VGA (video graphic array) connector. The display which we are using is standard 640 x 480 pixels. For imaging on the horizontal axis, we turn the video on from 0 till 639 pixels and for the vertical axis the video is on from 0 till 479 pixels as that is the main display. VGAsync was used for turning on the screen and pixel gen are used to create our desired screen. Following pin configuration is used to connect the VGA connector to the FPGA board



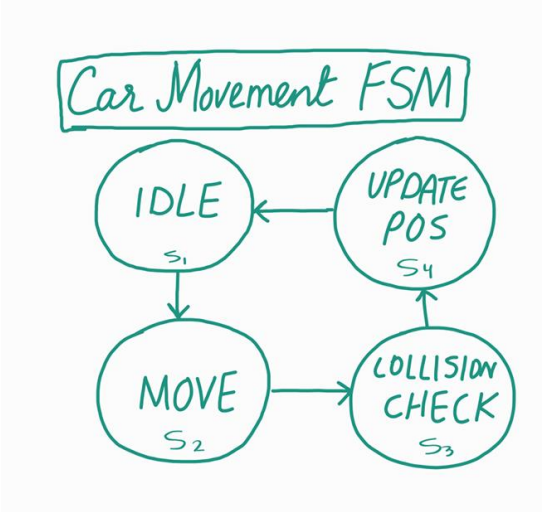
```
module vga_sync (
    input wire clk,          // 100MHz clock
    input wire reset,
    output reg hsync,        // Horizontal sync
    output reg vsync,        // Vertical sync
    output wire video_on,    // Active display area
    output wire [9:0] x,     // Pixel x-coordinate
    output wire [9:0] y,     // Pixel y-coordinate
    output wire p_tick       // Pixel clock tick (25MHz clock signal)
);
    // Timing constants
    localparam H_DISPLAY = 640; // Display width
    localparam H_FRONT   = 16;  // Front porch
    localparam H_SYNC    = 96;  // Sync pulse
    localparam H_BACK    = 48;  // Back porch
    localparam H_TOTAL   = 800; // Total horizontal pixels

    localparam V_DISPLAY = 480; // Display height
    localparam V_FRONT   = 10;  // Front porch
    localparam V_SYNC    = 2;   // Sync pulse
    localparam V_BACK    = 33;  // Back porch
    localparam V_TOTAL   = 525; // Total vertical lines

    // Internal registers
    reg [1:0] clk_div = 0;      // Clock divider for 25MHz
    reg [9:0] h_count = 0;      // Horizontal counter
    reg [9:0] v_count = 0;      // Vertical counter
```

```
module car_sprite(
    input wire [9:0] x,             // Current pixel x position
    input wire [9:0] y,             // Current pixel y position
    input wire [9:0] car_x,         // Car's top-left x position
    input wire [9:0] car_y,         // Car's top-left y position
    input wire [1:0] car_direction, // Direction car is facing
    input wire collision,           // Flag indicating if car is colliding
    input wire car_id,              // 0 for car1, 1 for car2 (different color
    output reg [3:0] sprite_red,    // Red component if pixel is part of sprit
    output reg [3:0] sprite_green, // Green component if pixel is part of spr
    output reg [3:0] sprite_blue,  // Blue component if pixel is part of spri
    output wire is_car_pixel        // High if current pixel is part of the ca
);
    // Car sprite dimensions
    parameter CAR_WIDTH = 12;
    parameter CAR_HEIGHT = 20;

    // Direction constants
    localparam DIR_UP = 2'd0;
    localparam DIR_RIGHT = 2'd1;
    localparam DIR_DOWN = 2'd2;
    localparam DIR_LEFT = 2'd3;

    // Calculate actual width and height based on rotation
    wire [4:0] actual_width = (car_direction == DIR_UP || car_direction == DI
    wire [4:0] actual_height = (car_direction == DIR_UP || car_direction == D
```

```
module ascii_rom(
    input clk,
    input wire [10:0] addr,
    output reg [7:0] data
    );

    (* rom_style = "block" *)    // Infer BRAM

    reg [10:0] addr_reg;

    always @(posedge clk)
        addr_reg <= addr;

    always @*
        case(addr_reg)
            // Begin non-printable ASCII characters (00 - 1f)
            // code x00 (nul) null byte, which is the all-zero pattern
            11'h000: data = 8'b00000000;    //
            11'h001: data = 8'b00000000;    //
            11'h002: data = 8'b00000000;    //
            11'h003: data = 8'b00000000;    //
            11'h004: data = 8'b00000000;    //
            11'h005: data = 8'b00000000;    //
            11'h006: data = 8'b00000000;    //
            11'h007: data = 8'b00000000;    //
            11'h008: data = 8'b00000000;    //
```

6. Control Block Design

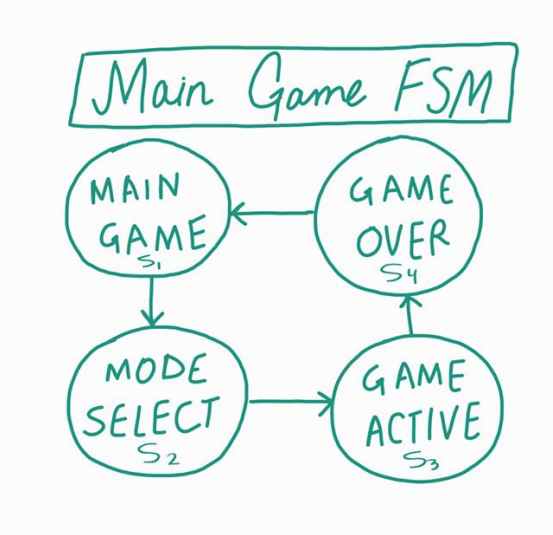
- **Game Logic FSM**:
  - Handles car movement, collisions, lap count, and timer
  - Uses internal counters for move delays
  - Supports car-car and border collision responses




```
4  module car_collision(
5      input wire [9:0] car_x,        // Car's top-left x position
6      input wire [9:0] car_y,        // Car's top-left y position
7      input wire [4:0] car_width,    // Car width
8      input wire [4:0] car_height,   // Car height
9      input wire [1:0] car_direction, // Direction car is facing
10     input wire p_tick,             // Pixel clock tick
11     input wire clk,                // System clock
12     input wire reset,              // Reset signal
13     input wire [9:0] x,            // Current pixel x
14     input wire [9:0] y,            // Current pixel y
15     input wire on_track,           // Signal from track generator if pixel is
16     input wire other_car_pixel,    // Flag indicating if current pixel is part
17     output reg collision,          // Collision detected
18     output reg car_car_collision, // Car-to-car collision detected
19     output reg border_up,          // Border detected above car
20     output reg border_down,        // Border detected below car
21     output reg border_left,        // Border detected left of car
22     output reg border_right        // Border detected right of car
23 );
24     // Car boundaries based on direction
25     wire [9:0] car_right, car_bottom;
26
27     // Adjust boundaries based on car direction
28     assign car_right = car_x + ((car_direction == 0 || car_direction == 2) ?
29     assign car_bottom = car_y + ((car_direction == 0 || car_direction == 2) ?
```

```
module game_logic(
    input clk,                    // 100MHz board clock
    input reset,
    input btnU, btnD, btnL, btnR,  // Debounced button inputs
    input player_switch,          // Switch to control which player moves (hig
    input game_active,            // Game active flag (disable movement if fal
    input border_up_p1,           // Border detected above car 1
    input border_down_p1,         // Border detected below car 1
    input border_left_p1,         // Border detected left of car 1
    input border_right_p1,        // Border detected right of car 1
    input border_up_p2,           // Border detected above car 2
    input border_down_p2,         // Border detected below car 2
    input border_left_p2,         // Border detected left of car 2
    input border_right_p2,        // Border detected right of car 2
    input car_car_collision_p1,   // Car-to-car collision detected for car 1
    input car_car_collision_p2,   // Car-to-car collision detected for car 2
    input ir_sensor_enable,       // NEW: IR sensor input to enable car moveme
    output reg [9:0] car1_x,      // X position of car 1
    output reg [9:0] car1_y,      // Y position of car 1
    output reg [1:0] car1_direction, // Direction car 1 is facing
    output reg [9:0] car2_x,      // X position of car 2
    output reg [9:0] car2_y,      // Y position of car 2
    output reg [1:0] car2_direction  // Direction car 2 is facing
);
```

- **Mode FSM**:
  - Switches between lap-based and time-based modes
  - Uses button inputs to select mode before gameplay begins



```
//======================================================
module lap_counter(
    input clk,
    input reset,
    input [9:0] car_x,        // Car's horizontal position
    input [9:0] car_y,        // Car's vertical position
    input [4:0] car_width,    // Car width
    input [4:0] car_height,   // Car height
    input [9:0] x,            // Current pixel x
    input [9:0] y,            // Current pixel y
    input p_tick,             // Pixel clock
    input on_start,           // Signal from track if current pixel is on sta
    output reg [7:0] laps     // Lap counter (supports up to 255 laps)
);
    // Car center for better lap detection
    wire [9:0] car_center_x = car_x + (car_width / 2);
    wire [9:0] car_center_y = car_y + (car_height / 2);

    // Flag to track if we're currently on the start line
    reg on_start_line;
    // Flag to prevent double counting within same crossing
    reg start_line_crossed;

    always @(posedge clk) begin
        if (reset) begin
            laps <= 0;
            on_start_line <= 0;
            start_line_crossed <= 0;
        end else if (p_tick) begin
```

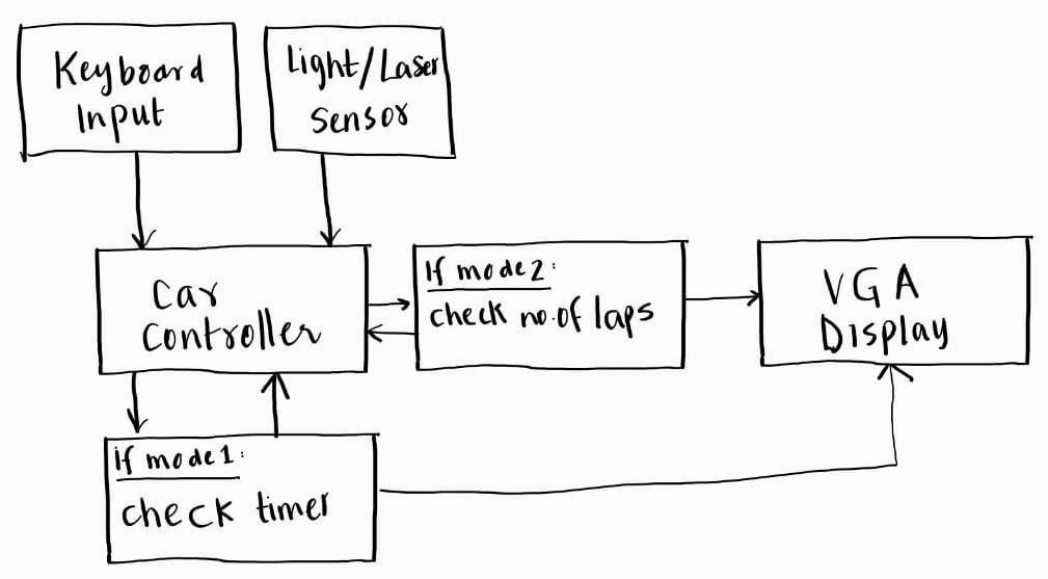
```
module game_timer(
    input clk,
    input reset,
    output reg [5:0] timer,    // 6-bit timer value (0 to 63 seconds; starting
    output reg game_over       // Flag that goes high when time is 0
);
    reg [26:0] counter;  // 27-bit counter to count 100M cycles (for 1 sec at

    initial begin
        timer = 63;
        counter = 0;
        game_over = 0;
    end

    always @(posedge clk) begin
        if (reset) begin
            timer <= 63;
            counter <= 0;
            game_over <= 0;
        end else if (!game_over) begin
            counter <= counter + 1;
            if (counter >= 100_000_000 - 1) begin // 1 second elapsed
                counter <= 0;
                if (timer > 0)
                    timer <= timer - 1;
                else
                    game_over <= 1;
            end
```

7. FSM Design

- FSM handles the following states: o IDLE: Waiting to start o RUNNING: Game in progress o FINISHED: Game over, display score
- Transitions depend on timer or laps and checkpoint logic.

In Overdrive, the game operates with a dynamic start screen that is presented upon loading. The transition from the start screen to the main game is initiated by pressing the green button. During gameplay, the user can navigate to the pause screen by pressing the

red button. Furthermore, we can conclude that our game exhibits a **Mealy** Machine behavior by considering user input as a factor in determining the next state. This design choice provides a more interactive and responsive gaming experience.



8. Results

The implementation of the car racing game on the Basys 3 FPGA board was successfully completed, with all primary features operating as intended. The system now utilizes a VGA display not only for the game interface but also for presenting the final scores clearly and effectively.

Player control has been enhanced through the integration of a joystick, enabling smooth and responsive car movement on the track. Mode selection—between Time-Based and Lap-Based gameplay—is managed using onboard push buttons, making the user interface intuitive and efficient.

A significant achievement was the accurate car movement and lap tracking, with the joystick allowing players to steer through a predefined track. The lap counter ensures that only laps completed through valid checkpoint sequences are counted, maintaining fairness and accuracy.

A countdown timer was successfully implemented using clock division and FSM-based control logic. When the timer expires (in time-based mode), gameplay stops and the final score is displayed on the VGA screen, giving players immediate feedback on their performance.

The system also includes physical pedal simulation through implemented light sensors, which detect foot motion to simulate acceleration and braking. This feature adds an immersive element to the game, moving it closer to a real arcade-style experience.

The FSM (Finite State Machine) controlling the game logic—managing states such as idle, running, paused, lap complete, and game over—performed reliably across all test cases, ensuring stable and predictable game behavior.

Overall, the project successfully combined digital logic principles with interactive gameplay. The physical arcade-style setup is under development, and the modular nature of the design allows for future enhancements such as obstacle addition, speed variation, and multiplayer support.

9. Challenges Faced

Track Design Difficulties:

- Initially planned to include 4 selectable tracks, but even a single track was hard to implement due to VGA memory and logic constraints.
- Designing valid paths, boundaries, and checkpoint logic in hardware was more complex than anticipated.

Graphical Display Issues:

- Creating and transitioning between multiple screens (main menu, mode selection, game screen, end screen) was time-consuming.
- Displaying final scores, countdown timers, and game states on VGA required intricate coordination between FSMs and pixel drawing logic.

Hardware & Electricity Constraints:

- Regular power outages disrupted progress, especially at home
- Only one team member had access to a PC for remote work, and it required repair before use.
- Majority of development had to be done on-campus under tight time slots.

Academic Pressure:

- Classes and quizzes continued until the final days of the semester, limiting working hours.
- Team members often stayed at university as late as 7 PM to work on the project.

Some members prioritized the project over exam preparation, sacrificing study time to meet deadlines.

10. Future Improvements

- Introduce dynamic obstacles and collision detection
- Implement difficulty levels by adjusting car speed
- Add sound effects and score animations
- Include pause/resume functionality via buttons
- Display time and score simultaneously on an LCD
- Complete the arcade model with mounted controls and sensors

11. Conclusion

This project reinforced key concepts in FSM design, modular hardware development, and timing control in Verilog. It provided practical experience in interfacing FPGA hardware with VGA output and handling real-time input from physical devices. The resulting game meets all functional goals and serves as a fun and technically rich example of applied digital design